Matlab-Cheatsheet für Signaltheorie (Version: Klausur WS 24/25 am 3.3.2025)

## Grundlagen:

### Erstellung von Vektoren

```
% Zeilenvektor mit Startwert A, Endwert B und Schritten von 1:
x = A:B
% Zeilenvektor mit Startwert A, Endwert B und Schritten von C:
x = A:C:B
% Zeitvektor mit Startwert 0s, Abtastrate fs (in Samples / s)
und Zeitdauer T (in s):
t = 0:1/fs:T-1/fs;
% oder
N = fs * T % Sample - Anzahl
t = (0:N-1)/fs;
```

### Standard-Signale:

```
% Sinus-Signal mit Frequenz f (in Hz) über Zeitvektor t (in s):
x = sin(2*pi*f*t) % Kosinus: cos(..)

% Rechtecksignal von 0.5s Länge, startet bei 0.3s
% Mit logischer Indizierung !
T = 1;                  % Zeitdauer in s
fs=10;                  % Abtastrate in Samples / s
t = 0:1/fs:1-1/fs       % Zeitvektor in s (dt = 1/fs)
x = zeros(size(t));     % Initialisierung mit 0en
x(t>=0.3 & t<0.8)=1.0;  % ler an richtigen Stellen setzen

% Sprungfunktion u(t) (=1 für t=>0) über Zeitvektor t
u = heaviside(t)

exp(t)                  % Exponentialfunktion
sqrt(t)                 % Quadratwurzel
log(t)                  % Natürlicher Logarithmus
t.^2;                   % Parabel
```

### Signal-Operationen:

```
% Ableiten (skaliert) Vektorlänge reduziert sich um 1!
x_D = diff(x)/dt;

% Integrieren (skaliert)
% 1. Stammfunktion (Ergebnis ist ein Vektor)
x_I = cumsum(x)*dt;
% 2. Bestimmtes Integral, Fläche (Ergebnis ist eine Zahl)
I = sum(x)*dt;

x_ = x + A;   % Konstante A aufaddieren
x_ = x * A;   % Konstante A heranmultiplizieren
y = x1 + x2;  % Element-weise Addition
y = x1 .* x2; % Element-weise Multiplikation
y = x1./x2;   % Element-weise Division
y = x.^2;     % Element-weise Quadrierung

% Diskrete Faltung
y = conv(x,h); % length(y)=length(x)+length(h)-1;

% Signal mit anhängenden 0en auffüllen (Anzahl: N0)
% x muss ein Zeilenvektor sein!
x_ = [x, zeros(1,N0)];

y = abs(x);   % Betrag
y = real(x);  % Realteil
y = imag(x);  % Imaginärteil
```

### Indizierung:

```
x = [0,2,4,6,8,10,12,14,16,18]
% Einzelner Wert
x_1 = x(1); % x_1 = 0

% Mehrere Werte (erstes, viertes und 7tes Element)
x_i = x([1,4,7]); % x_i = [0,6,12]

% Bereich (4tes bis einschl. 8tes Elemet)
x_i = x([4:8]);  % x_i = [6,8,10,12,14]

% Logische Indizierung:
x_neu = x(x>10); % x_neu = [12,14,16,18];
% Anwendung auf einen anderen Vektor (z.B. Zeitvektor)
t = (0:9); % [0,1,2,3,4,5,6,7,8,9]
% Alle Zeitwerte für die gilt x>10
t_neu = t(x>10); % t_neu = [6,7,8,9]
% alle x-Werte für die gilt t>5
x_neu_2 = x(t>5) % x_neu_2 = [12,14,16,18];
% Zusammengesetzte logische Ausdrücke
x_neu_3 = x(t>=2 & t<8); % x_neu_3 = [4,6,8,10,12,14]
```

### Elemente finden:

```
% Irgendein Signal x
x = [1,-4,-3,3,-2,2];

% Maximalwert
x_max = max(x) % x_max = 3
% Maximalwert und zugehöriger Index
[x_max, i_max] = max(x) % x_max = 3, i_max = 4

% Minimalwert
x_min = min(x) % x_min = -4
% Minimalwert und zugehöriger Index
[x_min, i_min] = min(x) % x_min = -4, i_max = 2

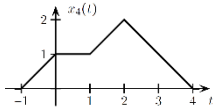
% Wo ist x gleich 3 ?
i_3 = find(x==3); % i_3 = 4;

% Wo ist x am nächsten an 2.23?
[abs_err,i_] = min(abs(x-2.23)); % i_ = 6, (abs_err = 0.23)
% abs_err will ich nicht wissen: Platzhalter ~
[~,i_] = min(abs(x-2.23)); % i_ = 6

% Auswertung von x bei t=2
t = 0:5; % Zeitvektor (fs=1)
x_t2 = x(t==2); % x_t2 = -3
```

### Funktions-Definitionen für Vektoren:

```
% Definition eines Zeitsignals als Funktion
x_fun = @(t) exp(-t);
% Definition von t
T_start = -2;     % Startzeitpunkt in s
T_stop = 5;       % Endzeitpunkt in s
fs=100;           % Abtastrate in Samples/s
t = T_start:1/fs:T_stop-1/fs;
% Auswertung:
x = x_fun(t);
```




```
% Bereichsweise definition (aus Übung 4, Aufgabe 1c)
x3_fun = @(t) (t>=-1 & t<0) .* (t+1) + (t>=0 & t<1) .* (1-t)

% Signaltransformation durch einfaches Einsetzen:
% x4(t) = x3(t) + 2 * x3(1/2*t -1)
x4 = x3_fun(t) + 2 * x3_fun(1/2*t-1)
```

## DFT / FFT:

x sei ein Signalvektor mit N Abtastwerten, die bei der Abtastrate fs (in Samples/s) abgetastet wurden.

### Realisierung der DFT des Signalvektors x

```
X = fft(x)
```

### Skalierung zur komplexen Fourier-Reihe / zweiseitiges Amplitudenspektrum:

```
% Skalierung:
X = fft(x) / N;

% Fourierkoeffizienten für +k und -k
% c_{k} = X[k]
cpk = X(k+1);  % Indizierung bei Matlab Startet bei 1
% c_{-k} = X[N-k]
cmk = X(N-k+1) % Indizierung bei Matlab startet bei 1

% Frequenzachse
df = fs / N;              % Frequenzauflösung
f = 0:df:fs-df;           % Positiver Frequenzvektor
f(f>=fs/2)=f(f>=fs/2)-fs; % Ab fs/2 -> 1*fs abziehen!

% Umdrehen: Zweiseitiges Amplitudenspektrum mit negativer und
positiver Hälfte
X_ds = fftshift(X);
f = fftshift(f);
```

### Einseitiges Amplitudenspektrum

```
% Skalierung:
X = abs(fft(x)) / N; % abs(): Betragsspektrum
X = X(1:floor(N/2+1));% Positive Achse. N gerade->fs/2 ist dabei
X_ss = X;
% N ist gerade
X_ss(2:end-1)=X_ss(2:end-1)*2;
% ODER: N ist ungerade:
X_ss(2:end)=X_ss(2:end)*2;

% Frequenzachse
df = fs / N;              % Frequenzauflösung
f = 0:df:floor(N/2)*df;   % Positiver Frequenzvektor
```

### Übertragungsfunktion

Übertragung / Filterung eines Signals mit einer definierten Übertragungsfunktion

```
H = @(f) ... % Definition einer Übertragungsfunktion über f
% z.B H = @(f) 1 ./ (1 + 1j * f)
df = fs / N;              % Frequenzauflösung
f_fft = 0:df:fs-df;           % Positiver Frequenzvektor
f_fft(f_fft>=fs/2)=f_fft(f_fft>=fs/2)-fs; % Ab fs/2 -> 1*fs
abziehen!

% x sei ein Eingangssignal der Länge N
y = real(ifft(H(f_fft) .* fft(x)));
% Auf Artefakte wegen Periodizität achten !
% Trick: x mit 0en verlängern!
% Equivalent zur Berechnung über Faltung!
```

## Plotten:

```
figure;                   % Erstellen einer figure
plot(t,x)                 % Signal x über t plotten
plot(t,x,"LineWidth",1.5) % mit definierter Linienstärke
plot(t,x,"LineStyle","--")% mit alternativem Linienstil
semilogx(t,x)             % X-Achse Logarithmisch
semilogy(t,x)             % Y-Achse Logarithmisch
loglog(t,x)               % Doppelt- Logarithmische Darstellung
% Stelle alle weiteren Plots im gleichen Fenster dar
hold on
% Stelle weitere Plots nicht im gleichen Fenster dar
hold off
legend(["x1(t)","x2(t)"]) % Legende (in diesem Fall 2 Einträge)
xlabel("t in s")          % X - Achsenbeschriftung
ylabel("Amplitude in V")  % Y - Achsenbeschriftung
xlim([a,b]);              % X Achsenlimits
ylim([a,b]);              % Y Achsenlimits
xticks([0.1,0.3,1.0])     % Ticks auf X - Achse manuell setzen
yticks([0.1,0.3,1.0])     % Ticks auf Y - Achse manuell setzen
% Labels für Ticks auf X - Achse manuell setzen
xticklabels(["0.1","0.3","1.0"])
% Labels für Ticks auf Y - Achse manuell setzen
yticklabels(["0.1","0.3","1.0"])

% "Insider-Tipp": Größe der figure auf definiertes Format in cm
skalieren:
set(gcf, 'Units', 'Centimeters', 'Position', [0 0 8.4 9],
'PaperPositionMode', 'auto')
```